Python Lambda Functions

Practice Questions

- Practice
- Solve
- Improve
- Build Logic
- Be a Better Coder

Small Functions Big Possibilities

Q1 Write a Python program using a lambda function to sort a list of strings based on their length (shortest to longest).

Problem Statement:

Given a list of strings, use a lambda function with sorted() to arrange the strings in increasing order of their length.

Sample Input:

```
words = ["apple", "kiwi", "banana", "fig", "grape"]
```

Expected Output:

```
['fig', 'kiwi', 'apple', 'grape', 'banana']
```

Code :

```
1  words = ["apple", "kiwi",
           "banana", "fig", "grape"]
2  sorted_words = sorted(words,
         key=lambda x: len(x))
3  print(sorted_words)
4
5
```

Explanation:

- The lambda function `lambda x: len(x)` returns the length of each string.
- sorted() arranges the strings based on the length returned by the lambda function.
- The strings are sorted from shortest to longest.

Lambda functions are often used with functions like sorted(), map(), filter(), etc.

Q2 Write a Python program using a lambda function to find all numbers greater than 50 from a list.

Problem Statement:

Given a list of numbers, use a lambda function with filter() to create a new list containing only the numbers greater than 50.

Sample Input:

```
nums = [10, 55, 23, 90, 67, 12, 80]
```

Expected Output:

```
[55, 90, 67, 80]
```

Code :

```
1  nums = [10, 55, 23, 90, 67, 12, 80]
2  result = list(filter(lambda x: x > 50,
                        nums))
3  print(result)
4
5
```

Explanation:

- The lambda function `lambda x: x > 50` checks if a number is greater than 50.
- filter() selects only the elements for which the condition is True.
- The result is converted to a list using list().

Lambda functions help you write small, one-line functions for simple tasks, making your code shorter and cleaner.

"Use small functions to solve big problems." – Satyarthi

Practice
Code
Learn
Grow

Python
Lambda Functions
Practice Questions

- Small Functions
- Smart Solutions
- Better Coding
- Brighter Future

Same
Concept
New
Possibilities

Q7 Write a Python program using a lambda function to check whether a number is even or odd.

Problem Statement:

Use a lambda function to check if a given number is even or odd and print the result.

Sample Input:

```
n = 17
```

Expected Output:

```
17 is Odd
```

Code :

```
1  n = 17
2  check = lambda x: "Even" if x % 2 == 0
                     else "Odd"
3  print(f"{n} is {check(n)}")
```

Explanation:

- The lambda function `lambda x: "Even" if x % 2 == 0 else "Odd"` checks whether the number is even or odd.
- If the number is divisible by 2 (remainder 0), it returns "Even", otherwise "Odd".
- The result is printed using an f-string.

- Lambda functions can contain conditional expressions, making them useful for simple decision-based tasks.

Q8 Write a Python program using a lambda function to find the maximum number from a list.

Problem Statement:

Given a list of numbers, use a lambda function with max() to find and print the largest number.

Sample Input:

```
nums = [12, 45, 7, 89, 23]
```

Expected Output:

```
89
```

Code :

```
1  nums = [12, 45, 7, 89, 23]
2  maximum = max(nums, key=lambda x: x)
3  print(maximum)
```

Explanation:

- The lambda function `lambda x: x` is used as the key in max() to compare the elements.
- max() returns the largest element from the list.
- The result is printed directly.

- Lambda functions can be used with functions like max(), min(), sorted(), etc., to perform operations on iterables easily.

"Code small, think big."

— Satyarthi

Keep Practicing | Keep Learning | Keep Growing

Python → A Brighter Tomorrow

Python Lambda Functions

Practice Questions

Practice
Code
Improve
Grow

Small Functions Big Possibilities

Q5 Write a Python program using a lambda function to convert all strings in a list to uppercase letters.

Problem Statement:

Given a list of strings, use a lambda function with map() to convert each string to uppercase.

Sample Input:

```
words = ["python", "lambda", "function", "code"]
```

Expected Output:

```
['PYTHON', 'LAMBDA', 'FUNCTION', 'CODE']
```

Code :

```
1  words = ["python", "lambda",
           "function", "code"]
2  upper_words = list(map(lambda x: x.upper(),
                          words))
3  print(upper_words)
4
```

Explanation:

- The lambda function lambda x: x.upper() converts each string to uppercase.
- map() applies the lambda function to all elements in the list.
- The result is converted to a list using list().

Lambda functions are useful for simple operations like string formatting, which can be written in one line.

Q6 Write a Python program using a lambda function to find the length of each string in a list and store the lengths in a new list.

Problem Statement:

Given a list of strings, use a lambda function with map() to get the length of each string and store the result in a new list.

Sample Input:

```
words = ["apple", "kiwi", "banana", "fig"]
```

Expected Output:

```
[5, 4, 6, 3]
```

Code :

```
1  words = ["apple", "kiwi",
           "banana", "fig"]
2  lengths = list(map(lambda x: len(x),
                      words))
3  print(lengths)
4
```

Explanation:

- The lambda function lambda x: len(x) returns the length of each string.
- map() applies the lambda function to all elements in the list.
- The result is converted to a list using list().

You can also use lambda functions with other built-in functions like len(), sum(), max(), etc.

"Small functions, big learning!" — Satyarthi

Python OOPS

Object Oriented Programming

Write better code. Model real-world problems.

- Classes
- Objects
- Inheritance
- Encapsulation
- Polymorphism
- Abstraction

Same Concept New Possibilities

1. What is OOPs?

- OOPs (Object Oriented Programming) is a programming approach that uses objects and classes to model real-world entities.
- It helps to make code more organized, reusable and easy to maintain.

"Everything in Python is an object."

2. Class and Object

- Class is a blueprint (template) for creating objects.
- Object is an instance of a class.

Class (Template) → creates → **Object** (Instance)

3. Constructor

- A constructor is a special method `__init__()` that is automatically called when an object is created.
- It is used to initialize the attributes of the object.

4. Example: Class, Object and Constructor

```
1  class Student:
2      def __init__(self, name, age):
3          self.name = name
4          self.age = age
5
6  # Creating object
7  s1 = Student("Rahul", 18)
8  print(s1.name)
9  print(s1.age)
```

Output:

```
Rahul
18
```

5. Class and Object in Real Life

Class (Car)

- Blueprint
- Attributes: color, model
- Methods: start(), stop()

Object (my_car)

- color = "Red"
- model = "SUV"
- start()
- stop()

"A class defines what an object has and can do. An object is a real instance with actual values."

6. Key Features of OOPs

1. **Encapsulation** – Hiding internal data and restricting direct access.
2. **Inheritance** – Reusing properties and methods from a parent class.
3. **Polymorphism** – Same method name, different behaviour.
4. **Abstraction** – Showing only essential details and hiding implementation.

7. Simple Example with Method

```
1  class Student:
2      def __init__(self, name, age):
3          self.name = name
4          self.age = age
5
6      def introduce(self):
7          print(f"Hi, I'm {self.name} and
           I'm {self.age} years old.")
8
9  s1 = Student("Sita", 20)
10 s1.introduce()
```

Output:

```
Hi, I'm Sita and
I'm 20 years old.
```

8. Advantages of OOPs

- Modular and organized code
- Code reusability
- Easier to maintain and debug
- Helps in solving real-world problems
- Widely used in large projects and industries

9. Where OOPs is Used?

- Web development (Django, Flask)
- Game development
- Desktop applications
- Mobile applications
- Real-world system modeling (e.g., Bank, Hospital, Library)

10. Quick Recap

- ✓ Class = blueprint
- ✓ Object = instance
- ✓ `__init__()` = constructor
- ✓ Methods = functions inside class
- ✓ OOPs = Encapsulation + Inheritance + Polymorphism + Abstraction

"Good code is not just about making it work, but about making it better for the future."

— Satyarthi

Python
Class Attributes and Object Attributes

Learn
Practice
Code
Build
Grow

Small Concepts Big Possibilities

Same Class, Different Attributes, Different Uses

1. Class Attribute

- A class attribute is a variable that is shared by all objects of the class.
- It is defined inside the class but outside any method.
- It is the same for every object (unless changed using the class name).
- Accessed using ClassName.attribute or object.attribute (but best practice is using class name).

"One copy, shared by all objects."

2. Object Attribute

- An object attribute is a variable that is specific to each object.
- It is usually defined inside the __init__() method using self.
- Each object can have different values for its object attributes.
- Accessed using object.attribute.

"Each object has its own copy."

3. Syntax

Class Attribute

```
class ClassName:
    class_attr = value
    def __init__(self):
        self.obj_attr = value
```

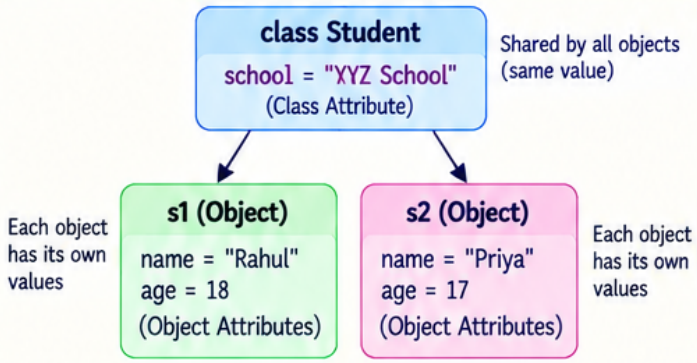
Object Attribute

```
class ClassName:
    class_attr = value
    def __init__(self):
        self.obj_attr = value
```

4. Example Code

```
1  class Student:
2      school = "XYZ School"  # Class attribute
3      def __init__(self, name, age):
4          self.name = name     # Object attribute
5          self.age = age       # Object attribute
6
7  # Creating objects
8  s1 = Student("Rahul", 18)
9  s2 = Student("Priya", 17)
10
11 print(s1.school)  # Accessing class attribute
12 print(s2.school)
13 print(s1.name, s1.age)  # Accessing object attributes
```

5. Visual Representation



6. Output

```
XYZ School
XYZ School
Rahul 18
```

Key Point:
Class attribute is same for all objects, while object attributes can be different for each object.

7. Modifying Attributes

Modify Class Attribute

```
Student.school = "ABC School"
print(s1.school)  # ABC School
print(s2.school)  # ABC School
```

Modify Object Attribute

```
s1.name = "Rohan"
print(s1.name)   # Rohan
print(s2.name)   # Priya

s1.age = 19
print(s1.age)    # 19
```

8. Important Notes

- Class attributes are shared by all objects.
- Object attributes are unique for each object.
- If an object attribute has the same name as a class attribute, the object attribute will take preference (for that object).
- Class attributes are useful for storing common information (e.g., school name, company name).
- Object attributes are useful for storing individual data (e.g., name, age, marks).

"Understand the basics well, and coding becomes easier." — By Satyarthi

Learn
Practice
Grow

Python
Object Methods

Methods define what objects can do!

- ☐ Understand
- ☐ Write Code
- ☐ Solve Problems
- ☐ Build Confidence

Same
Concept
New
Possibilities

1. What are Object Methods?

- A method is a function defined inside a class that operates on the object's data (attributes).
- Methods define the behaviour of an object.
- They are called using the object name and dot (.) operator.
- The first parameter of a method is usually self, which refers to the current object.

"Methods tell an object what to do."

2. Syntax of an Object Method

```
class ClassName:
    def method_name(self, parameters):
        # method body
        # can access and modify object attributes
        pass
```

- def is used to define a method.
- self refers to the current object.
- Methods can take additional parameters (optional).
- They are called using: object_name.method_name()

3. Example: Object Method

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"Hi, I am {self.name} and
        I am {self.age} years old.")

# Creating object
s1 = Student("Rahul", 18)
s1.introduce()  # Calling object method
```

Student Object

4. Output

```
Hi, I am Rahul and
I am 18 years old.
```

5. How It Works?

1. s1.introduce() calls the method.
2. self refers to s1 (the current object).
3. The method accesses the object's attributes (self.name and self.age).
4. It performs an action (printing the message).

6. Example: Method with Parameters

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

r = Rectangle(5, 3)
print("Area:", r.area())  # Calling method
```

Output:

```
Area: 15
```

7. Key Points

- ✓ Methods are functions inside a class.
- ✓ They operate on object data (attributes).
- ✓ self refers to the current object.
- ✓ Methods can have parameters.
- ✓ They are called using the dot (.) operator.
- ✓ Methods can return values.
- ✓ Each object can call the same method with different data.

8. Real Life Analogy

An object is like a mobile phone.
Methods are like the functions you can do with it — call(), message(), camera(), etc.

"Objects have data, methods give them life."

— By Satyarthi

Python Constructors

Initialize objects automatically!

- [ ] Understand
- [ ] Practice
- [ ] Write Code
- [ ] Solve Problems
- [ ] Build Confidence

Same Concept New Possibilities

1. What is a Constructor?

- A constructor is a special method that is automatically called when an object is created.
- It is used to initialize the attributes (variables) of the object.
- In Python, the constructor method is `__init__()`.
- It does not have a return type (not even void).
- It is called only once, when the object is created.

> *"Constructor initializes the object as soon as it is created."*

2. Syntax

```
class ClassName:
    def __init__(self, parameters):
        # initialize attributes
        self.attribute = value
        self.attribute = value
```

- `__init__()` is the constructor method.
- `self` refers to the current object.
- It can take parameters (optional).
- It is called automatically when an object is created.

3. Example: Constructor with Parameters

```
1  class Student:
2      def __init__(self, name, age):
3          self.name = name
4          self.age = age
5
6  s1 = Student("Rahul", 18)
7  print(s1.name)
9  print(s1.age)
```

Object created with constructor

Output:

```
Rahul
18
```

4. Example: Constructor without Parameters

```
1  class Book:
2      def __init__(self):
3          self.title = "Unknown"
4          self.pages = 0
5
6  b1 = Book()
7  print(b1.title)
8  print(b1.pages)
```

Output:

```
Unknown
0
```

5. Key Points

- Constructor is defined using `__init__()`.
- It is called automatically when an object is created.
- It is used to initialize object attributes.
- It can have parameters or no parameters.
- It does not return any value.
- It is called only once per object.

6. Real Life Analogy

A constructor is like the manufacturing setup of a car. When a car is made, certain things (engine, wheels, color, etc.) are set automatically. Similarly, when an object is created, the constructor initializes its attributes.

7. Common Mistakes

- Writing the constructor name incorrectly (e.g., `_init_()`).
- Forgetting to use `self`.
- Expecting a return value from the constructor.
- Calling the constructor manually (not required).

> *"A good start (constructor) leads to a well-built object."* — By Satyarthi

Keep Practicing | Keep Learning | Keep Growing

Python → A Brighter Tomorrow

Python Constructors

Practice Questions

- Practice
- Think
- Write Code
- Check Output
- Improve

Same Concept New Possibilities

Q1 Create Objects Using a Constructor

Write a Python program to create a class Employee with a constructor that initializes the attributes name, emp_id, and salary. Create two objects with different values and display their details using a method show_details().

Code:

```
1  class Employee:
2      def __init__(self, name, emp_id, salary):
3          self.name = name
4          self.emp_id = emp_id
5          self.salary = salary
6
7      def show_details(self):
8          print(f"Name: {self.name}")
9          print(f"Employee ID: {self.emp_id}")
10         print(f"Salary: {self.salary}")
11
12 # Create two objects here
```

Your Task:

1. Create two Employee objects with different values.
2. Call the show_details() method for each object.
3. Observe and write the output.

Expected Output (Your Output):

Q2 Constructor Without Parameters

Write a Python program to create a class Student with a constructor that does not take any parameters. Inside the constructor, assign default values to the attributes name, age, and course. Create an object and display the details using a method show().

Code:

```
1  class Student:
2      def __init__(self):
3          self.name = "Satyam"
4          self.age = 18
5          self.course = "B.Sc. Biotechnology"
6
7      def show(self):
8          print(f"Name: {self.name}")
9          print(f"Age: {self.age}")
10         print(f"Course: {self.course}")
11
12 # Create object and call method here
```

Your Task:

1. Create an object of the Student class.
2. Call the show() method.
3. Write the output.

Expected Output (Your Output):

"Practice constructors today, build better programs tomorrow."

— By Satyarthi

Python Constructors

Practice Questions (Set 2)

Practice
Code
Learn
Improve
Grow

Same Concept New Possibilities

Q3 Book Class with Constructor

Write a Python program to create a class Book with a constructor that initializes the attributes title, author, and price. Create two Book objects with different values and display their details using a method show_info().

Requirements:

1. Create a class Book with a constructor that takes title, author, and price as parameters.
2. Define a method show_info() to display the book details in a readable format.
3. Create two objects of the class with different values.
4. Call the show_info() method for each object.

Sample Output (Format):

```
Book Details:
Title  : The Alchemist
Author : Paulo Coelho
Price  : 250.0

Book Details:
Title  : Ikigai
Author : Hector Garcia
Price  : 300.0
```

"A good book is a good friend."

Q4 BankAccount Class with Constructor

Write a Python program to create a class BankAccount with a constructor that initializes the attributes account_number, holder_name, and balance. Create two objects with different values and display their details using a method display_account().

Requirements:

1. Create a class BankAccount with a constructor that takes account_number, holder_name, and balance as parameters.
2. Define a method display_account() to show the account details.
3. Create two BankAccount objects with different values.
4. Call the display_account() method for each object.

Sample Output (Format):

```
Account Details:
Account Number : 1234567890
Holder Name    : Rahul Kumar
Balance        : 15000.0

Account Details:
Account Number : 9876543210
Holder Name    : Priya Singh
Balance        : 25000.0
```

"Save Today for a Better Tomorrow."

"Code your ideas, create a better future." — By Satyarthi

Python Constructors

Answers (Set 2)

Same Concept New Possibilities

Q3 Book Class with Constructor

Write a Python program to create a class Book with a constructor that initializes the attributes title, author, and price. Create two Book objects with different values and display their details using a method show_info().

Answer:

```python
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price

    def show_info(self):
        print("Book Details:")
        print("Title :", self.title)
        print("Author:", self.author)
        print("Price :", self.price)

# Create two objects
b1 = Book("The Alchemist", "Paulo Coelho", 250)
b2 = Book("Ikigai", "Hector Garcia", 300)

# Call method for each object
b1.show_info()
print()  # for better readability
b2.show_info()
```

Output:

```
Book Details:
Title : The Alchemist
Author: Paulo Coelho
Price : 250

Book Details:
Title : Ikigai
Author: Hector Garcia
Price : 300
```

"Read More Grow More"

Q4 BankAccount Class with Constructor

Write a Python program to create a class BankAccount with a constructor that initializes the attributes account_number, holder_name, and balance. Create two objects with different values and display their details using a method display_account().

Answer:

```python
class BankAccount:
    def __init__(self, account_number,
                 holder_name, balance):
        self.account_number = account_number
        self.holder_name = holder_name
        self.balance = balance

    def display_account(self):
        print("Account Details:")
        print("Account Number:", self.account_number)
        print("Holder Name   :", self.holder_name)
        print("Balance       :", self.balance)

# Create two objects
a1 = BankAccount("1234567890", "Rahul Kumar", 15000)
a2 = BankAccount("9876543210", "Priya Singh", 25000)

# Call method for each object
a1.display_account()
print()  # for better readability
a2.display_account()
```

Output:

```
Account Details:
Account Number: 1234567890
Holder Name   : Rahul Kumar
Balance       : 15000

Account Details:
Account Number: 9876543210
Holder Name   : Priya Singh
Balance       : 25000
```

"Save Today for a Better Tomorrow"

"Practice today, build a better tomorrow." — By Satyarthi